

BÁO CÁO LAB 5

TẤN CÔNG BRUTE FORCE VÀ PHÒNG CHỐNG TRÊN FORM ĐĂNG NHẬP

Họ tên: UNG VẮNG ĐẠI LÂM MSSV: 2380601202 Lớp: 23DATA1

Hình thức: Cá nhân. Mã nguồn: github.com/UngVangDaiLam/ATw-CSDL_Lab (thư mục lab5).

⚠ **Lưu ý:** Script brute force chỉ trở tới <http://localhost:5000> (app demo tự dựng), nhằm hiểu cách phòng chống. Không dùng để dò mật khẩu trên hệ thống của người khác — đó là hành vi vi phạm pháp luật.

1. Môi trường thực hành

App demo **Node.js + Express** (`demo-app/`), tài khoản cài sẵn **user1 / Pass123**. Server có hai endpoint để so sánh: **/api/login-unprotected** (không giới hạn) và **/api/login-protected** (giới hạn số lần sai + CAPTCHA). Script tấn công **Python + requests** (`attack/brute\_force.py`) dùng một wordlist nhỏ gồm các mật khẩu phổ biến, trong đó có Pass123.

Chạy app: `cd demo-app` → `npm install` → `npm start` (<http://localhost:5000>). Chạy tấn công: `cd attack` → `python brute\_force.py unprotected` hoặc `python brute\_force.py protected`.

2. Quy trình tấn công brute force

- Script lần lượt lấy từng mật khẩu trong wordlist, gửi **POST** tới endpoint đăng nhập bằng thư viện requests.
- Với mỗi lần thử, đọc phản hồi của server: đăng nhập **sai** → **HTTP 401** `{success:false}`; đăng nhập **đúng** → **HTTP 200** `{success:true}`.
- Khi gặp phản hồi thành công, script dừng và in ra mật khẩu tìm được.

Kết quả trên endpoint KHÔNG bảo vệ: script dò ra mật khẩu **Pass123** sau **10 lần thử** trong chưa đầy một giây (Hình 2). Vì server không giới hạn, kẻ tấn công có thể thử hàng nghìn mật khẩu liên tục.

```
$ python brute_force.py unprotected
[*] Bắt đầu brute force -> /api/login-unprotected (user: user1)
[*] Wordlist: 12 mật khẩu

[ 1/12] thử '123456' -> ✗ sai (HTTP 401)
[ 2/12] thử 'password' -> ✗ sai (HTTP 401)
[ 3/12] thử 'admin' -> ✗ sai (HTTP 401)
[ 4/12] thử 'qwerty' -> ✗ sai (HTTP 401)
[ 5/12] thử '111111' -> ✗ sai (HTTP 401)
[ 6/12] thử 'user1' -> ✗ sai (HTTP 401)
[ 7/12] thử 'letmein' -> ✗ sai (HTTP 401)
[ 8/12] thử 'abc123' -> ✗ sai (HTTP 401)
[ 9/12] thử 'iloveyou' -> ✗ sai (HTTP 401)
[10/12] thử 'Pass123' -> ✓ THÀNH CÔNG! Mật khẩu = 'Pass123'

[+] Tìm ra mật khẩu sau 10 lần thử.
```

Hình 2. Brute force thành công trên endpoint không bảo vệ

3. Thiết lập biện pháp phòng chống

Trên endpoint **/api/login-protected**, server tự theo dõi số lần đăng nhập sai theo từng tài khoản và áp dụng:

- Giới hạn số lần sai:** đếm số lần sai; đạt **5 lần** → **khóa tạm thời 10 phút** (trả HTTP 429). Đăng nhập đúng sẽ reset bộ đếm.
- CAPTCHA:** sau **3 lần sai**, server bắt buộc giải CAPTCHA (demo dùng phép cộng; thực tế dùng **Google reCAPTCHA**). Request thiếu/sai CAPTCHA bị từ chối (HTTP 403), nên script tự động không thể gửi mật khẩu tiếp.

Đoạn mã chính (rút gọn) trong `server.js`:

```
if (rec.lockUntil > now) return res.status(429)... // đang bị khóa if (rec.fails >= 3 &&
captcha !== session.captcha) // bắt CAPTCHA return res.status(403).json({ needCaptcha:
true }); if (sai mật khẩu) { rec.fails++; if (rec.fails>=5) lock 10'; }
```

4. Kết quả thử nghiệm và đánh giá

Chạy lại chính script brute force đó trên endpoint có bảo vệ: script **bị chặn ngay ở lần thử thứ 3** vì server yêu cầu CAPTCHA (Hình 3). Nếu tiếp tục sai sẽ bị khóa tài khoản 10 phút. Hai biện pháp hoạt động đúng như thiết kế.

```
$ python brute_force.py protected

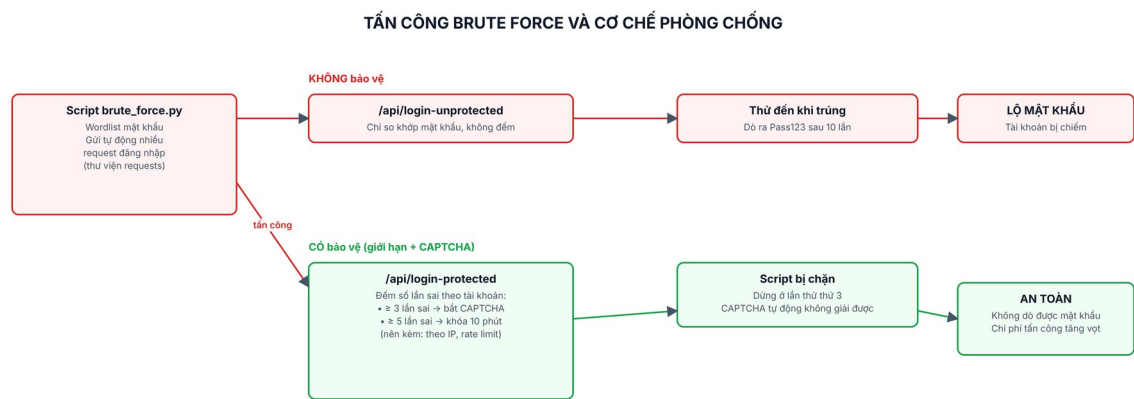
[*] Bắt đầu brute force -> /api/login-protected (user: user1)
[*] Wordlist: 12 mật khẩu

[ 1/12] thử '123456' -> ✖ sai (HTTP 401)
[ 2/12] thử 'password' -> ✖ sai (HTTP 401)
[ 3/12] thử 'admin' -> 🚫 YÊU CẦU CAPTCHA: Sai username hoặc mật khẩu. (Đã sai 3/5 lần)

[-] Dừng ở lần thử 3: server bắt CAPTCHA, script tự động không vượt qua được.
```

Hình 3. Cùng script bị chặn trên endpoint có bảo vệ

Tiêu chí	Không bảo vệ	Có bảo vệ
Số lần thử được	Không giới hạn	Tối đa 3 lần rồi vướng CAPTCHA, 5 lần thì khóa
Kết quả brute force	Tìm ra mật khẩu (Pass123)	Bị chặn, không dò được
Chi phí tấn công	Rất thấp, tự động hoàn toàn	Rất cao: mỗi lần phải giải CAPTCHA, lại bị khóa



Hình 1. Sơ đồ tấn công brute force và cơ chế phòng chống

5. Bài học và đề xuất nâng cao

- Form chỉ kiểm mật khẩu mà không giới hạn số lần thử thì **chắc chắn bị brute force**; mật khẩu càng yếu càng nhanh lộ.
- Giới hạn số lần thử + CAPTCHA** làm chi phí tấn công tăng vọt, là tuyến phòng thủ hiệu quả và dễ triển khai.
- Trong thực tế nên bổ sung: đếm theo **IP** kết hợp tài khoản (dùng express-rate-limit), lưu trạng thái ở **Redis**; mật khẩu băm **bcrypt**; chính sách mật khẩu mạnh; **xác thực 2 lớp (2FA)**; cảnh báo và ghi log khi có nhiều lần sai bất thường; dùng **Google reCAPTCHA** thay CAPTCHA tự chế.

6. Kết luận

Thử nghiệm cho thấy brute force có thể dò ra mật khẩu yếu chỉ trong vài lần thử khi server không có cơ chế giới hạn. Sau khi thêm giới hạn số lần đăng nhập sai và CAPTCHA, cùng một script tấn công bị chặn ngay từ những lần thử đầu. Đây là minh chứng rõ ràng rằng kiểm soát tần suất đăng nhập là biện pháp bảo mật thiết yếu cho mọi form đăng nhập.